VI - TP 2 Parcours de dictionnaire

1 Sur le réseau social TipTop, on s'intéresse au nombre de « like » des abonnés. Les données sont stockées dans un dictionnaire où les clés sont les pseudos et les valeurs correspondantes sont les nombres de « like » comme ci-dessous :

Code python
```python
likes = {'Bob': 102, 'Ada': 201, 'Charline': 103, 'Daniel': 201}
```

1. Écrire une fonction `nb_likes` qui étant donné un dictionnaire `likes` et le `nom` d'une personne, renvoie le nombre de likes de `nom`. Si `nom` n'est pas présent dans `likes`, alors la fonction renverra `0`.

Code python
```python
def nb_likes(likes, nom):
    """ {str:int}, str -> int
    Renvoie le nombre de likes de nom """
    pass
```

Code python
```python
print(nb_likes(likes, "Ada"))
print(nb_likes(likes, "Esther"))
```

Résultat
```
201
0
```

2. Écrire une fonction `ajoute_likes` qui étant donné un dictionnaire `likes`, le `nom` d'une personne et un entier `l` ajoute `l` au nombre de likes de `nom`.

Code python
```python
def ajoute_likes(likes, nom, l):
    """ {str: int}, str, int -> None
    Ajoute l au nombre de likes de nom """
    pass
```

Code python
```python
print(likes)
ajoute_likes(likes, 'Bob', 42)
ajoute_likes(likes, 'Esther', 128)
print(likes)
```

Résultat
```
{'Bob': 102, 'Ada': 201, 'Charline': 103, 'Daniel': 201}
{'Bob': 144, 'Ada': 201, 'Charline': 103, 'Daniel': 201, 'Esther':
↪ 128}
```

3. Écrire une fonction `top_likes` qui prend en argument un dictionnaire `likes` non-vide dont les clés sont des chaines de caractères et les valeurs associées des entiers positifs ou nuls, et qui renvoie le `nom` d'une personne dans `likes` qui possède le plus de likes.

   En cas d'égalité entre les deux personnes les plus "likées", on en choisira l'une des deux sans contrainte particulière.

Code python

```python
def top_likes(likes):
    """ {str: int} -> str
    Renvoie le nom d'une personne possédant le plus de likes """
    nom = ''
    maxi = 0
    # À compléter
```

Code python

```python
print(top_likes(likes))
```

Résultat

```
Ada
```

4. Modifier le code de la fonction `top_likes` en une fonction `top_likes_mieux` de telle sorte que `top_likes_mieux(likes)` renvoie la **liste** des personnes qui possèdent le plus de likes.

Code python

```python
def top_likes_mieux(likes):
    """ {str: int} -> [str]
    Renvoie la liste des personnes possédant le plus de likes """
    noms = []
    maxi = 0
    # À compléter
```

Code python

```python
print(top_likes_mieux(likes))
```

Résultat

```
['Ada', 'Daniel']
```

2 Les résultats d'une élection sont stockés dans une liste de la forme :

Code python

```python
votes = ["Ali", "Bob", "Mél", "Ali", "Ali", "Mél"]
```

Le nom des candidat.e.s n'est pas connu à l'avance. On suppose que tous les candidat.e.s reçoivent un nombre de vote différent.

1. Écrire une fonction `compte_voix` qui prend en argument une liste de `votes` et qui renvoie un dictionnaire de telle sorte que :

- les clés du dictionnaires correspondent aux noms des personnes pour qui il a été voté lors de cette élection ;
- à chaque clé `k` du dictionnaire on associe le nombre de voix que `k` a reçu lors de l'élection.

Code python

```python
def compte_voix(votes):
    """ [str] -> {str: int}
    Renvoie le decompte des voix des votes """
    # À compléter
```

Code python

```python
decompte = compte_voix(votes)
print(decompte)
```

Résultat

```
{'Ali': 3, 'Bob': 1, 'Mél': 2}
```

2. Écrire une fonction `pourcentage` qui prend en argument un dictionnaire `decompte` et le `nom` d'une personne et qui renvoie le pourcentage (arrondi au centième) des votes pour `nom` dans le `decompte`.

Code python

```python
def pourcentage(decompte, nom):
    """ {str: int}, str -> float
    Renvoie le pourcentage de votes obtenu par nom dans décompte """
    # À compléter
```

Code python

```python
print(pourcentage(decompte, 'Mél'))
```

Résultat

```
33.33
```

3. Écrire une fonction `resultat_second_tour` qui prend en argument un dictionnaire `decompte` correspondant aux votes ayant eu lieu au second tour et qui renvoie le nom de la personne ayant obtenu le plus de votes au second tour : il s'agit alors de la personne élue.

Code python

```python
def resultat_second_tour(decompte):
    """ {str: int} -> str
    Renvoie le nom de la personne ayant obtenu le plus de votes """
    decompte['] = 0
    meilleur = ''
    # À compléter
```

Code python

```python
print(resultat_second_tour(decompte))
```

Résultat

```
Ali
```

4. On souhaite pouvoir gérer informatiquement le premier tour.

a. Écrire une fonction `deux_meilleurs` qui prend en argument un dictionnaire `decompte` et qui renvoie un dictionnaire possédant les clés `'1er'` et `'2nd'` associées respectivement au nom des personnes arrivées première et deuxième dans le décompte des votes de l'élection.

Code python

```python
def deux_meilleurs(decompte):
    """ {str: int} -> {str: str}
    Renvoie un dictionnaire correspondant aux deux votes
    ↪ majoritaires """
    decompte['] = 0
    podium = {'1er': '', '2nd': ''}
    # À compléter
```

Code python

```python
print(deux_meilleurs(decompte))
```

Résultat

```
{'1er': 'Ali', '2nd': 'Mél'}
```

b. Écrire une fonction `resultat_premier_tour` qui étant donné un dictionnaire `decompte` correspondant aux votes ayant eu lieu au premier tour renvoie :

- une liste constituée du nom de la personne élue si une personne a obtenu plus de 50% des voix au premier tour ;
- une liste constituée des noms des deux personnes présentes au second tour si personne n'a obtenu plus de 50% des voix au premier tour.

Code python

```python
def resultat_premier_tour(decompte):
    """ {str: int} -> [str]
    Renvoie la ou les personnes gagnantes au premier tour """
```

Code python

```python
print(resultat_premier_tour(decompte))
```

Résultat

```
['Ali', 'Mél']
```

3 Un professeur de NSI décide d'utiliser python pour gérer informatiquement les notes de ses élèves. On suppose que toutes les notes ont le même coefficient. On utilise pour stocker les notes un dictionnaire : les clés sont les prénoms des élèves de la classe, la valeur associée à la clé `k` la liste des notes obtenues par l'élève.

Code python

```python
notes = {
    "Aba": [15, 16, 12],
    "Bouba": [19, 20, 20, 20],
    "Cem": [9, 13, 14, 15]
}
```

1. Écrire une fonction `ajoute_note` qui prend en argument un dictionnaire de `notes` et un `prenom` et un entier `n` et qui ajoute la note `n` à la liste de notes de `prenom`.

   Attention. Les clés du dictionnaire peuvent être amenées à évoluer lors de l'arrivée d'un nouvel élève. Ainsi, il sera nécessaire de prendre en compte le cas où `prenom` n'est pas une clé de `notes`.

Code python

```python
def ajoute_note(notes, prenom, n):
    """ {str: [int]}, str, int -> None
    Ajoute n à la liste de notes de prenom """
    pass
```

Code python

```python
ajoute_note(notes, "Aba", 17)
ajoute_note(notes, "David", 14)
print(notes)
```

Résultat

```
{'Aba': [15, 16, 12, 17], 'Bouba':
↪ [19, 20, 20, 20], 'Cem': [9, 13,
↪ 14, 15], 'David': [14]}
```

2. La fin du trimestre arrivant, le professeur souhaite obtenir des informations sur les moyennes de ses élèves. Vous écrirez avec soin la signature et la documentation de chacunes des fonctions suivantes.

   a. Écrire une fonction `moyenne_eleve` qui prend en argument un dictionnaire de `notes`, le `prenom` d'un élève, et qui renvoie sa moyenne actuelle.

   On rappelle que toutes les notes sont affectées du même coefficient.

   b. Écrire une fonction `moyenne` qui prend en argument un dictionnaire de `notes` et qui renvoie un dictionnaire dont :

   - les clés sont les prénoms des élèves de la classe ;
   - la valeur associée à la clé `k` est la moyenne de l'élève `k`

   c. Écrire une fonction `moyenne_classe` qui prend en argument un dictionnaire de `notes` et qui renvoie la moyenne pondérée des moyennes des élèves. La moyenne de `prenom` sera pondérée par le nombre de notes associées à `prenom` dans `notes`.

   Par exemple, dans l'exemple ci-dessus, la moyenne de David a pour coefficient 1, la moyenne des autres élèves a pour coefficient 4.